Sistemas Operativos II

Práctica 2 — Informe Opcionales

Sincronización avanzada con hilos POSIX (pthreads)

Asignatura:	Sistemas Operativos II
Titulación:	Grado en Ingeniería Informática
Curso:	2024/25
Fecha:	10 de marzo de 2026

Universidad de Santiago de Compostela

Índice

1. Ejercicio Opcional 1: Productor-Consumidor con Hilos **2**
1.1. Descripción arquitectónica 2
1.2. Análisis de resultados 2

2. Ejercicio Opcional 2: N Productores y M Consumidores **3**
2.1. Diseño y control de concurrencia 3
2.2. Protocolo de Terminación en Redes M:N 3
2.3. Análisis de resultados 3

3. Ejercicio Opcional 3: Arquitectura en Pipeline **5**
3.1. Descripción del Pipeline Lineal 5
3.2. Transformación de datos (`next_alpha`) 5
3.3. Propagación en cascada y resultados 5

1. Ejercicio Opcional 1: Productor-Consumidor con Hilos

1.1. Descripción arquitectónica

A diferencia de los apartados obligatorios donde se utilizaban procesos pesados comunicados mediante `mmap()`, en esta versión (`opc1_threads.c`) se instancian dos hilos dentro de un único proceso utilizando la biblioteca POSIX Threads (`pthread`).

La memoria es compartida de forma natural ya que todos los hilos residen en el mismo espacio de direcciones virtuales. Esto elimina la necesidad de utilizar ficheros mapeados en memoria.

Semáforos sin nombre (`sem_init`)

En lugar de crear semáforos nombrados en el sistema de archivos del kernel mediante `sem_open()`, se utilizan semáforos anónimos basados en memoria mediante `sem_init(&sem, 0, valor)`. El segundo argumento a `0` indica que el semáforo será compartido exclusivamente entre los hilos del proceso actual, evitando dejar recursos huérfanos en el sistema.

1.2. Análisis de resultados

La ejecución evidencia tres fases temporales claramente delimitadas por los `sleep()` situados en el código:

1. **Productor más rápido:** El buffer se llena rápidamente hasta su capacidad máxima (`top=10`).
2. **Consumidor más rápido:** El consumidor agota los recursos del buffer llegando a `top=0`.
3. **Fase aleatoria:** Ambos procesos se bloquean alternativamente.

La salida final corrobora el correcto uso del semáforo mutex para la sección crítica:

```
[MAIN] === Vocales producidas ===
  a/A=8  e/E=6  i/I=8  o/O=5  u/U=3
[MAIN] === Vocales consumidas ===
  a/A=8  e/E=6  i/I=8  o/O=5  u/U=3
[MAIN] OK: vocales producidas == vocales consumidas.
```

2. Ejercicio Opcional 2: N Productores y M Consumidores

2.1. Diseño y control de concurrencia

La generalización del problema a múltiples productores y consumidores (`opc2_multi.c`) introduce complicaciones de sincronización adicionales. Además de proteger el buffer, es necesario proteger otros recursos compartidos:

- **`mutex_prod`:** Protege el acceso al puntero de lectura del fichero `input.txt` compartido entre los N productores.
- **`mutex_fin`:** (Junto con variables `atomic_int`) Protege los contadores globales de elementos producidos y consumidos para gestionar la terminación.
- **`mutex_vocales_[prod|cons]`:** Evita condiciones de carrera al actualizar los contadores estadísticos de vocales.

2.2. Protocolo de Terminación en Redes M:N

El problema clásico en redes de varios productores y consumidores es evitar que los consumidores queden bloqueados en un `sem_wait(&llenas)` infinito una vez que los productores han terminado.

La solución implementada utiliza una combinación de variables atómicas y *broadcasting* mediante semáforos:

1. El hilo principal (`main`) espera mediante `pthread_join()` a que todos los productores terminen.
2. Se activa el flag atómico `produccion_fin`.
3. El hilo principal ejecuta `sem_post(&llenas)` M veces (una por cada consumidor).
4. Los consumidores atrapados en espera se despiertan, evalúan la condición de finalización y propagan la señal si es necesario antes de hacer `break`.

2.3. Análisis de resultados

La ejecución solicitada de 3 productores y 2 consumidores reporta una sincronización perfecta para 120 elementos en total (40 elementos por productor):

```
[MAIN] 3 productor(es), 2 consumidor(es)
[MAIN] Total a producir/consumir: 120 items
...
[PROD-0] i=39  producido='c'
[PROD-0] i=39  insertado  top=5  producidos_total=120
[CONS-0] retirado='c'  top=4  consumidos_total=116
...
```

```
[MAIN] === Resumen ===
[MAIN] Producidos : 120
[MAIN] Consumidos : 120
[MAIN] Vocales producidas: a=12 e=12 i=8 o=8 u=4
[MAIN] Vocales consumidas: a=12 e=12 i=8 o=8 u=4
[MAIN] OK: producidos == consumidos, vocales coinciden.
```

3. Ejercicio Opcional 3: Arquitectura en Pipeline

3.1. Descripción del Pipeline Lineal

El archivo `opc3_pipeline.c` implementa un flujo de datos secuencial a través de P hilos, conectados por $P-1$ buffers intermedios de tamaño fijo. Cada hilo intermedio actúa simultáneamente como **consumidor** de la etapa anterior y como **productor** de la etapa siguiente.

La topología permite el **máximo grado de concurrencia**: al disponer de arrays independientes de semáforos (`g_vacias[k]`, `g_llenas[k]`, `g_mutex[k]`), el hilo 1 puede estar escribiendo en el buffer 1 al mismo tiempo que el hilo 2 escribe en el buffer 2. No existe un *Global Interpreter Lock* ni bloqueo general del sistema.

3.2. Transformación de datos (`next_alpha`)

Para evidenciar el flujo a través de las etapas intermedias, cada hilo aplica una función de cifrado de rotación unitaria cíclica (tipo César +1) exclusivamente a los caracteres alfabéticos.

3.3. Propagación en cascada y resultados

Para terminar la ejecución, el Productor Origen (Hilo 0) inserta un carácter centinela nulo (`'\0'`) que cada hilo intermedio detecta y propaga a la etapa adyacente antes de terminar su propia ejecución.

Se muestra un pipeline configurado con $P = 4$ etapas (2 etapas intermedias de transformación):

```
[MAIN] Pipeline de 4 etapas, 3 buffer(es) intermedios.
[MAIN] Cada caracter pasa por 2 transformacion(es) next_alpha.
[MAIN] 80 items a producir.
...
[Hilo  0] i= 0  leido='E'
[Hilo  0->buf 0] insertado='E'  top=1
[buf 0->Hilo  1] retirado='E'  top=0
[Hilo  1] 'E' -> 'F'
[Hilo  1->buf 1] insertado='F'  top=1
[buf 1->Hilo  2] retirado='F'  top=0
[Hilo  2] 'F' -> 'G'
[Hilo  2->buf 2] insertado='G'  top=1
[buf 2->Hilo  3] retirado='G'  top=0
[Hilo  3] consumido='G'
...
[MAIN] === Resumen del pipeline ===
[MAIN] Transformaciones aplicadas: 2 (next_alpha x2)
[MAIN] Vocales en origen  (hilo 0):   a=8 e=6 i=8 o=5 u=3
```

```
[MAIN] Vocales al final   (hilo 3):  a=1 e=3 i=2 o=2 u=0
[MAIN] (Las vocales pueden cambiar tras transformaciones:
   correcto)
[MAIN] Vocales totales origen=30, finales=8
[MAIN] OK: se procesaron exactamente 80 caracteres.
```

Discrepancia de Vocales

A diferencia de los ejercicios anteriores, el conteo de vocales inicial y final no coincide. Esto es estrictamente correcto y esperado, ya que las transformaciones intermedias (ej. $e \to f \to g$) convierten vocales en consonantes y viceversa. La validación de integridad recae en que los caracteres iniciales y procesados totales son exactamente 80.